DAY 3 PROJECT REPORT

Touch-Screen Fingerprint System using ESP32, AS608 (DY50) Fingerprint Sensor and 3.5" SPI TFT with Touch

Project: DLC Lab Project | Date: 3 November 2026 | Day: 3

Item	Details
Prepared by	Md Arif Mahmud Oyion, Md Rakibul Islam, Niloy Bormon, Sheum Ahmed (American International University-Bangladesh)
Date / Day	Tuesday, 3 November 2026 (Day 3)
Main controller	ESP32 DevKit V1 (ESP32-D0WD-V3, revision v3.1, dual core, 240 MHz)
Day 3 status	COMPLETED. Display, touch and fingerprint sensor all working together through a touch menu (Enroll, Delete, Test, Erase All).

1. Objective of the Day

The aim of Day 3 was to connect three separate modules to one ESP32 board and make them work together as a small fingerprint system that is controlled entirely from a touch screen:

- Identify the three modules and read their pin labels from the boards.
- Wire the 3.5" SPI TFT display (with touch panel) and the DY50 (AS608) fingerprint sensor to the ESP32 safely.
- Test every device one at a time (display, touch, fingerprint) before combining them.
- Write one program in which the user can register a new finger, delete a finger and test a finger using only the touch screen.
- Record every problem faced, how it was diagnosed and how it was solved, so that the work can be repeated without the same mistakes.

2. Apparatus and Components Used

2.1 Hardware

No.	Item	Purpose / key facts
1	ESP32 DevKit V1 (30-pin, micro-USB)	Main microcontroller. 3.3 V logic. Pins are silk-screened D5, D18, D19, TX2, RX2 and so on. Only one 3V3 pin and two GND pins.
2	DY50 fingerprint module (AS608 chip)	Optical fingerprint sensor. 6-pin connector: GND, RX, TX, 3V3, T-OUT, T-3V3. UART 57600 baud, 3.3 V. Stores templates in its own flash memory.
3	3.5" SPI TFT, 480x320, with resistive touch (XPT2046 touch controller)	14-pin header: T_IRQ, T_DO, T_DIN, T_CS, T_CLK, SDO(MISO), LED, SCK, SDI(MOSI), DC/RS, RESET, CS, GND, VCC. Solder jumper J1 selects the power route. Display controller was driven with the ILI9488 driver.
4	Digital multimeter	Voltage checks (DC volts) on VIN, 3V3 and module supply pins; continuity (beep) test on jumper wires and for shorts, always with

No.	Item	Purpose / key facts
		power OFF.
5	Breadboard	Gives a shared 3.3 V rail and a shared GND rail, because the ESP32 has only one 3V3 pin and three wires need it.
6	Male-to-male and male-to-female jumper wires	All signal and power connections.
7	Micro-USB data cable and USB port / 5 V adapter	Programming and power supply. Must be a data-capable cable of good quality.
8	Computer (macOS) with Arduino IDE	Writing, compiling and uploading the code; Serial Monitor at 115200 baud.
9	Stylus or pen tip (optional)	Pressing the resistive touch screen accurately during calibration.

2.2 Software and libraries

Software	Version / note
Arduino IDE	With ESP32 board package (core 3.3.12 shown in the upload log). Board: "ESP32 Dev Module".
TFT_eSPI (Bodmer)	Display and touch driver. Configured only through User_Setup.h.
Adafruit Fingerprint Sensor Library	Talks to the AS608 over UART (Serial2).
Preferences (built in)	Saves the touch calibration numbers to ESP32 flash.

3. Component Identification

Before wiring anything, the pin names printed on each board were read from photographs. This avoids guessing and prevents connecting a supply wire to a signal pin.



Figure 1: ESP32 DevKit V1 - pins printed as D5, D18, D19, D23, TX2, RX2, VIN, 3V3, GND.

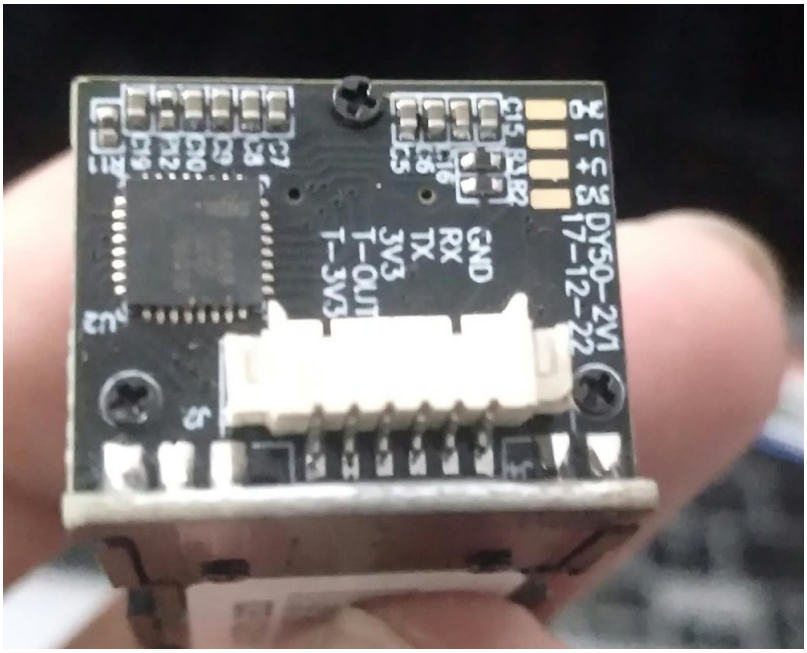


Figure 2: DY50 fingerprint module (AS608). Silkscreen shows GND, RX, TX, 3V3, T-OUT, T-3V3.

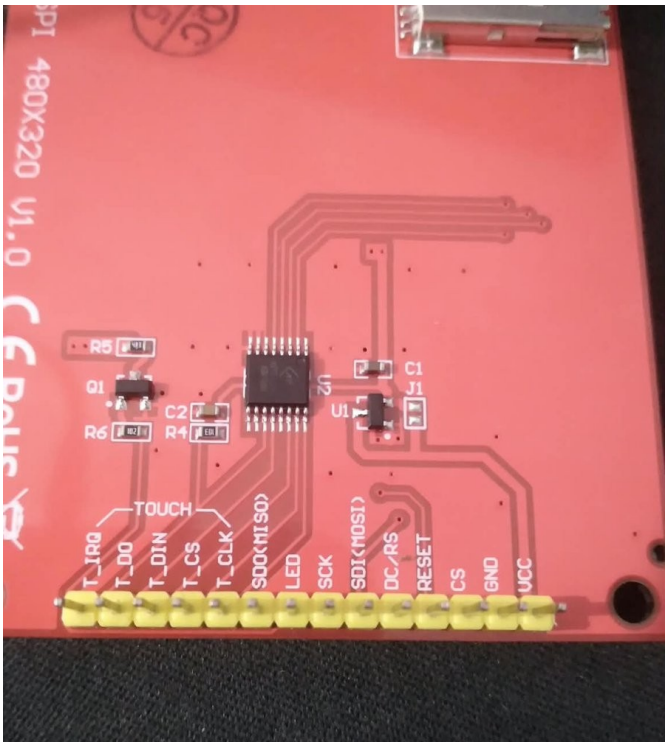


Figure 3: Back of the 3.5" TFT. The 14 header pins are labelled; J1 is the power jumper next to regulator U1.

Important facts learned from the labels

- **TX and RX on the DY50 are named from the module's point of view,** so they cross over: DY50 TX goes to ESP32 RX2 and DY50 RX goes to ESP32 TX2.
- The ESP32 has only one 3V3 pin, but three wires need 3.3 V (TFT LED, DY50 3V3, DY50 T-3V3). A breadboard rail is required.
- The TFT display and the touch controller share one SPI bus (SCK, MOSI, MISO) but have separate chip-select pins (CS for the display, T_CS for the touch).
- The AS608 must be powered with 3.3 V so its TX signal is safe for the ESP32 input pin.

4. Circuit Connections

4.1 Wiring diagram

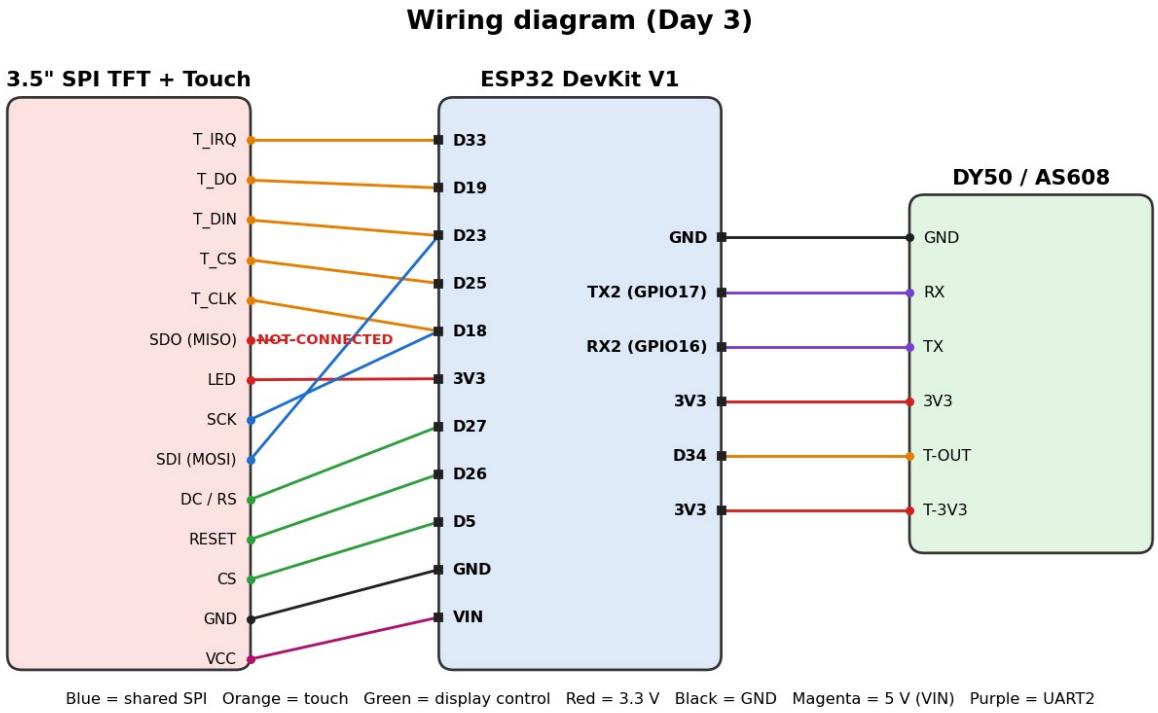


Figure 4: Complete wiring between the TFT, the ESP32 and the DY50 module. TFT SDO(MISO) is intentionally left unconnected.

4.2 Master pin map (ESP32 label to module pin)

ESP32 pin (as printed)	GPIO	Connects to	Function
VIN	5 V	TFT VCC	Display power (J1 open)
3V3	3.3 V	TFT LED, DY50 3V3, DY50 T-3V3	Backlight, sensor power, touch-ring power (via breadboard rail)
GND	0 V	TFT GND, DY50 GND	Common ground
D5	5	TFT CS	Display chip select
D18	18	TFT SCK and T_CLK	Shared SPI clock
D19	19	T_DO only	SPI MISO (touch data out). TFT SDO(MISO) left empty
D23	23	TFT SDI(MOSI) and T_DIN	Shared SPI MOSI
D26	26	TFT RESET	Display reset
D27	27	TFT DC/RS	Data / command
D25	25	TFT T_CS	Touch chip select
D33	33	TFT T_IRQ	Touch interrupt (optional, not used in code)
RX2	16	DY50 TX	Fingerprint UART receive
TX2	17	DY50 RX	Fingerprint UART transmit

ESP32 pin (as printed)	GPIO	Connects to	Function
D34	34	DY50 T-OUT	Finger-present signal (optional, input-only pin)

4.3 Power plan

- TFT VCC is fed with 5 V from VIN. The TFT has its own 3.3 V regulator, so this spares the small regulator on the ESP32. Jumper J1 must be OPEN (not bridged with solder).
- TFT LED (backlight) is fed from 3V3.
- The fingerprint sensor (3V3 and T-3V3) stays on 3.3 V only, never on VIN.
- Fallback if unsure about J1: bridge J1 and feed TFT VCC from 3V3 instead. Never mix the two methods.

5. Multimeter Voltage and Continuity Test

Use the multimeter before uploading code and again whenever something does not light up. Set the dial to DC volts (V with a straight line) on the 20 V range. Put the black probe on a GND pin and the red probe on the point being tested. The table lists what a healthy circuit should read at each test point; a reading far outside the range points to a wiring or power fault.

Measure between	Expected reading	What a wrong reading means
ESP32 VIN and GND (USB connected)	about 4.5 - 5.2 V	Weak USB cable or port; VIN wire loose
ESP32 3V3 and GND	about 3.2 - 3.4 V	ESP32 regulator overloaded or short on the 3.3 V rail
TFT VCC and TFT GND	about 4.5 - 5.2 V	VCC wire not on VIN, or GND not connected
TFT LED and TFT GND	about 3.3 V	Backlight wire missing from the 3V3 rail (Error 2)
DY50 3V3 and DY50 GND	about 3.3 V	Sensor supply missing from the 3V3 rail (Error 2)
DY50 T-3V3 and DY50 GND	about 3.3 V	Touch-ring supply missing
DY50 T-OUT to GND (optional)	low at rest, about 3.3 V when a finger touches the ring	T-3V3 not powered or T-OUT wire loose

Result on the bench: once the 3.3 V rail was connected (Error 2), the display backlight and the fingerprint sensor started working, and the whole system ran normally for the rest of the session, which shows the supply rails were within their working range.

Continuity (beep) test - power OFF, USB unplugged

- Touch the probes to the two ends of each jumper wire. A beep means the wire is good.
- Check there is NO beep between 3V3 and GND, and none between VIN and GND. A beep there means a short circuit; do not power the board until it is removed.
- Check that T_DO really reaches D19 and that TFT SDO(MISO) reaches nothing.

Never use the resistance or continuity mode while the circuit is powered, and never put the probes across 5 V while the meter is in current (A) mode.

6. Software Setup

1. Install the Arduino IDE and add the ESP32 board package. Select "ESP32 Dev Module" and the port /dev/cu.usbserial-0001.
2. Install the libraries TFT_eSPI and Adafruit Fingerprint Sensor Library with the Library Manager.
3. Replace the contents of TFT_eSPI/User_Setup.h with the custom file given in Appendix A. In User_Setup_Select.h only the line #include <User_Setup.h> must be active.
4. Fully quit and reopen the Arduino IDE so the library is rebuilt with the new settings.
5. Upload test sketches in this order: display, raw touch, touch drawing, fingerprint, then the final combined program.

Board details read from the upload log: ESP32-D0WD-V3, revision v3.1, Wi-Fi and Bluetooth, dual core 240 MHz, 40 MHz crystal. Final program size about 24 percent of flash and 6 percent of RAM, so there is plenty of room for future features.

7. Work Log (Step by Step)

Step	What was done	Result
1	Photographed the three boards and read the pin labels. Identified the DY50 as an AS608 module and the TFT as a 480x320 SPI display with touch.	Pin names confirmed.
2	Prepared the wiring plan, power plan and pin map (Section 4).	Plan ready.
3	Wired the ESP32 and TFT and installed libraries.	Compile error (Error 1).
4	Replaced the default User_Setup.h with the custom ESP32 setup.	Code compiled and uploaded.
5	Display and fingerprint sensor gave no sign of power.	The 3V3 wires were missing (Error 2). Connected via breadboard rail.
6	Display powered on, but touch did not respond.	Raw touch test showed the fault; SDO(MISO) removed (Error 3).
7	Touch drawing test uploaded and calibrated.	Touch working (Figures 5 to 8). Slight lag and jitter noted (Error 4).
8	Final combined program with touch menu uploaded.	Enroll, test and delete working through the touch screen.

8. Errors Faced and How They Were Solved

This section is the most important part of the day. Each error is listed with the exact symptom, the real cause, the fix and the lesson.

Error 1: Compile error - TFT_eSPI has no member named getTouch

Symptom: Arduino IDE stopped with five errors: "'class TFT_eSPI' has no member named 'getTouch'", and the same for setTouch and calibrateTouch.

Cause: The library was still using the stock factory User_Setup.h. In that file the ILI9341 driver was active, the pins were ESP8266 pins (PIN_D6, PIN_D7, PIN_D8) and the line #define TOUCH_CS was commented out. The touch functions are only compiled into the library when TOUCH_CS is defined, so they simply did not exist.

Fix: Checked that User_Setup_Select.h included only User_Setup.h, then replaced the whole User_Setup.h with the custom file in Appendix A (ILI9488 driver, ESP32 pins, TOUCH_CS 25). Quit and reopened the Arduino IDE and compiled again - the errors disappeared.

Lesson: TFT_eSPI is configured only by User_Setup.h, not by the sketch. After editing it always restart the IDE. A quick test is to place #ifndef TOUCH_CS / #error at the top of the sketch.

Error 2: Display and fingerprint sensor showed no sign of power

Symptom: Upload finished successfully ("Hard resetting via RTS pin") but the TFT stayed dark and the fingerprint sensor showed no light.

Cause: The 3.3 V connections had not been made. The ESP32 has only one 3V3 pin, so the TFT LED, DY50 3V3 and DY50 T-3V3 wires could not all be plugged into it directly, and they had been left unconnected. The upload log was not the problem - it only shows that flashing succeeded.

Fix: Joined the three 3V3 wires using the breadboard 3.3 V rail, checked GND was common, and measured the rail with the multimeter. After restarting the ESP32 the display lit up.

Lesson: When nothing lights up, check power with the multimeter first. A successful upload says nothing about the wiring of other modules.

Error 3: Touch did not respond (shared MISO line)

Symptom: The display worked, but touching the screen gave no response and calibration looked stuck.

Diagnosis: Uploaded a raw touch test (Appendix D) that prints x, y and pressure to the Serial Monitor. The values were not changing correctly when the screen was pressed.

Cause: The TFT SDO(MISO) pin and the touch T_DO pin were both wired to D19. The ILI9488 display does not release (tri-state) the MISO line when its chip select is high, so it kept driving the line and corrupted the data coming from the touch controller. The warning for exactly this is written in the TFT_eSPI User_Setup.h file next to the ILI9488 driver line.

Fix: Removed the wire from TFT SDO(MISO) and left that pin empty. T_DO stays on D19. The display only needs to receive data, so nothing is lost. After this the x, y and pressure values changed correctly when pressed.

Lesson: On a shared SPI bus, a device that does not release MISO breaks every other device on the bus. Test touch with raw values first, before blaming calibration.

Error 4: Touch drawing was slow, dotted and slightly jittery

Symptom: In the touch drawing test (Figures 5 to 8) the yellow dots were spaced apart and the lines wobbled a little.

Cause: Two reasons. The test program redrew the X/Y text and printed to the Serial Monitor for every touch sample, which slowed the loop. Also, a resistive screen is pressure based and has natural electrical noise.

Fix: In the final program the screen is not redrawn on every sample. A button press must be detected twice in a row at almost the same position, the pressure threshold is set to 400, and buttons are large (110 x 90 pixels), so small jitter does not matter. Calibration is done by pressing the arrows firmly and is saved to flash.

Lesson: Design touch screens for resistive panels with big buttons and simple filtering. Press with a fingertip or stylus, not a light touch.

Design problem found during the day: calibration blocked everything

In the first combined sketch the touch calibration ran before the fingerprint sensor was started and waited forever for a tap. When the touch did not work, nothing else could be tested. The solution was to test each device in its own small sketch first, and to save the calibration values to flash in the final program so the calibration is done only once.

Summary table of errors

No.	Problem	Root cause	Solution
1	getTouch / setTouch / calibrateTouch not found	Default User_Setup.h: TOUCH_CS commented out, wrong driver and ESP8266 pins	Custom User_Setup.h, restart IDE
2	No power indication on display and sensor	3V3 wires not connected (only one 3V3 pin on ESP32)	Breadboard 3.3 V rail, check with multimeter
3	Touch not responding	TFT SDO(MISO) and T_DO both on D19; ILI9488 does not release MISO	Leave TFT SDO(MISO) unconnected
4	Dotted, jittery touch drawing	Redraw and Serial print on each sample; resistive panel noise	Debounce, threshold 400, large buttons, saved calibration

9. Results

9.1 Touch test

The touch drawing test was calibrated and then used to draw on the screen. The four photographs below were taken at the end of the touch test. The drawings follow the finger position, and the four coloured corner squares, the CLEAR, RECAL and ON/OFF buttons are visible.

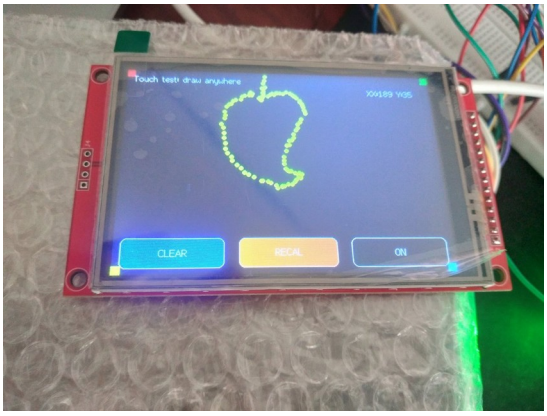


Figure 5: Single stroke drawing (shape follows the finger).



Figure 6: Longer strokes across the screen.

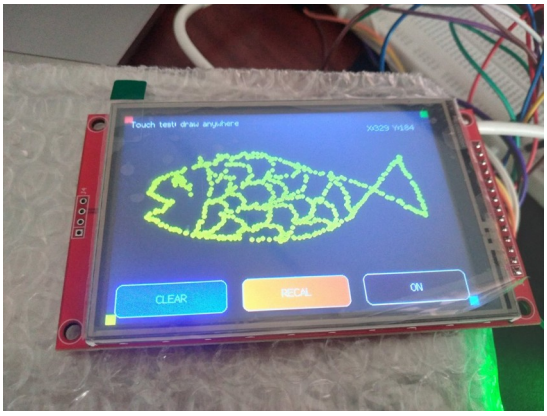


Figure 7: Dense drawing, X/Y coordinates shown at top right.

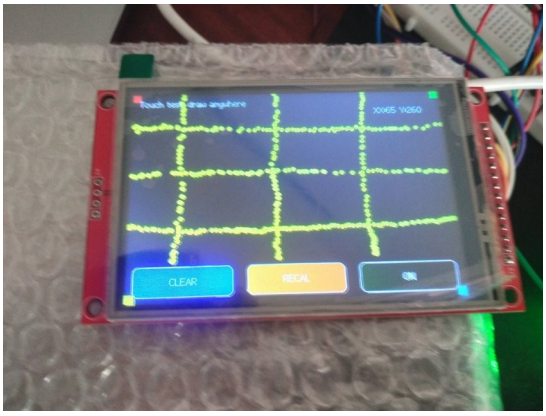


Figure 8: Grid lines drawn to check accuracy (photo rotated for reading).

9.2 Test results

Test	Pass condition	Result
Display colour test	Red, green, blue fill the screen	Pass
Raw touch test	x, y and pressure change when the screen is pressed	Pass (after removing SDO wire)
Touch drawing test	Dots appear under the finger; buttons work	Pass (Figures 5 to 8)
Touch calibration saved	No calibration needed at next boot	Pass
Fingerprint sensor found	Sensor answers on Serial2 at 57600 baud	Pass
ENROLL from touch menu	Finger stored, count increases	Pass
TEST from touch menu	Enrolled finger shows MATCH with ID; other finger shows ACCESS DENIED	Pass
DELETE from touch menu	Finger removed, later test is denied	Pass
ERASE ALL (double tap)	All templates wiped	Pass

The final result was confirmed at the end of the session: the whole system works properly, with the fingerprint functions controlled from the touch screen.

9.3 How the final program works

Button	Action
ENROLL	Finds the first free ID, asks the user to place a finger, remove it, and place the same finger again, then stores the template. Shows "Enrolled! ID = x".
DELETE	User places the finger to remove; the sensor finds the matching ID and deletes it.
TEST	Scans a finger and searches the database. Shows MATCH with ID and score (green) or ACCESS DENIED (red).
ERASE ALL	Needs two taps within 3 seconds, then deletes every stored fingerprint.

During any "place finger" step the user can touch the screen to cancel. The header always shows how many fingerprints are stored. Each wait has a 10-second timeout.

10. Precautions to Take Before and During Work

Electrical safety

- Always unplug USB before changing any wire. Wire first, check, then power.
- **Never connect 5 V to any ESP32 GPIO pin** or to the fingerprint sensor pins. The ESP32 pins are 3.3 V only.
- Keep the fingerprint sensor on 3V3 only. Do not power it from VIN.
- Check jumper J1 on the TFT before powering: open if VCC is on 5 V, bridged only if VCC is on 3.3 V. Do not mix.
- Make sure all grounds (ESP32, TFT, sensor) are joined together.
- Use the multimeter to confirm voltages (Section 5) and to find shorts with the continuity test before first power-up.
- If a board gets hot, smells or a component discolours, unplug USB immediately.

Handling and wiring

- Touch a grounded metal object before handling bare boards to discharge static. Do not touch the pins or chips while powered.
- Do not use TX0 and RX0 for other devices; they are the USB serial lines and break uploads.
- Cross the UART wires: sensor TX to ESP32 RX2, sensor RX to ESP32 TX2.
- Push jumper wires in firmly and check each wire against the printed label; header pins are close together and wires slip to the neighbouring pin.
- Do not press the resistive screen with sharp metal objects; use a fingertip or a stylus. Do not press hard on the glass of the fingerprint sensor.
- Use a good data-capable USB cable and a stable port. Thin cables cause resets and a dim display.

Software and testing

- Test one device at a time: ESP32 alone, display, touch, fingerprint, then combine.
- Edit TFT_eSPI settings only in User_Setup.h and restart the Arduino IDE after every change.
- Keep the Serial Monitor open at 115200 baud to read messages.
- Save the touch calibration numbers; write down the final pin map and keep a copy of the working code.
- Do not leave the sketch waiting for input before other hardware is initialised.
- Enrolled fingerprints live in the sensor flash; ERASE ALL cannot be undone.

11. Observations and Learning Outcomes

- Reading the silkscreen labels carefully and writing a pin map before wiring saves a lot of debugging time.
- Several SPI devices can share SCK, MOSI and MISO, but each needs its own chip-select and none may hold MISO when not selected.
- A compile error about missing functions can come from a library configuration file and not from the sketch.
- "No light" or "no response" is best diagnosed with a multimeter and a small isolated test sketch.
- The AS608 stores fingerprints in its own memory, so they remain after power-off and the ESP32 only needs to send commands.

- Resistive touch needs calibration and firm pressure; saving calibration data makes the project easy to use.

12. Plan for Next Day (Day 4) - Suggestions

- Add names for each ID and display "Welcome <name>" on a match.
- Drive a relay or electronic lock from a free GPIO on a successful match (use a transistor or relay module with its own supply).
- Use the sensor T-OUT pin (D34) so the sensor scans only when a finger is on the ring.
- Log attendance with time using Wi-Fi/NTP and store records on a server or SD card.
- Move the circuit from the breadboard to a soldered board or a case with a proper 5 V supply.

13. Conclusion

On Day 3 the ESP32, the AS608 (DY50) fingerprint sensor and the 3.5" SPI touch display were identified, wired, tested one by one, and finally combined into one working system. Four main problems were met and solved: a library configuration error, missing 3.3 V connections, a shared MISO conflict that stopped the touch panel, and a laggy touch test. The final program lets a user enroll, test and delete fingerprints and erase all of them using only the touch screen. The notes and precautions in this report make it possible to repeat the work safely and quickly.

Appendix A: TFT_eSPI User_Setup.h (final)

Replace the full contents of User_Setup.h with this. If the screen is white or garbled, change ILI9488_DRIVER to ST7796_DRIVER.

```
#define USER_SETUP_INFO "User_Setup"

// ---- Driver (use only ONE) ----
#define ILI9488_DRIVER       // if screen stays white/garbled, try ST7796_DRIVER

// ---- Pins (ESP32 DevKit V1) ----
#define TFT_MISO 19
#define TFT_MOSI 23
#define TFT_SCLK 18
#define TFT_CS    5
#define TFT_DC   27
#define TFT_RST  26
#define TOUCH_CS 25          // enables getTouch / setTouch / calibrateTouch

// ---- Fonts ----
#define LOAD_GLCD
#define LOAD_FONT2
#define LOAD_FONT4
#define SMOOTH_FONT

// ---- SPI speeds ----
#define SPI_FREQUENCY       20000000
#define SPI_READ_FREQUENCY  16000000
#define SPI_TOUCH_FREQUENCY  2500000
```

Appendix B: Final program (touch menu + fingerprint)

File name: fingerprint_touch_app.ino. Touch calibration is saved to flash on the first run.

```
/*
  ESP32 + 3.5" SPI TFT (XPT2046 touch) + AS608 / DY50 fingerprint sensor
  Touch menu: ENROLL / DELETE / TEST / ERASE ALL

  Wiring (as agreed):
    TFT: CS=5  DC=27  RST=26  MOSI=23  SCK=18  T_CS=25  T_CLK=18  T_DIN=23  T_DO=19
         TFT SDO(MISO) left UNCONNECTED
    DY50 TX -> GPIO16 (RX2)     DY50 RX -> GPIO17 (TX2)     DY50 3V3 -> 3V3

  Libraries: TFT_eSPI (User_Setup.h with TOUCH_CS 25), Adafruit Fingerprint Sensor Library
  Touch calibration is saved to flash, so you calibrate only once.
*/

#include <TFT_eSPI.h>
#include <Adafruit_Fingerprint.h>
#include <Preferences.h>

#define FP_RX_PIN 16
#define FP_TX_PIN 17
#define FP_BAUD   57600

TFT_eSPI tft;
Adafruit_Fingerprint finger(&Serial2);
Preferences prefs;
uint16_t calData[5];

struct Button {
  int x, y, w, h;
  const char* label;
```

```
  uint16_t color;
};

Button btns[4] = {
  {  10, 200, 110, 90, "ENROLL",    TFT_DARKGREEN},
  { 130, 200, 110, 90, "DELETE",    TFT_MAROON},
  { 250, 200, 110, 90, "TEST",      TFT_NAVY},
  { 370, 200, 100, 90, "ERASE ALL", TFT_BROWN},
};

bool eraseArmed = false;
unsigned long eraseArmTime = 0;

// ============================================================== UI helpers
void drawButton(int i, const char* labelOverride = nullptr) {
  Button &b = btns[i];
  tft.fillRoundRect(b.x, b.y, b.w, b.h, 8, b.color);
  tft.drawRoundRect(b.x, b.y, b.w, b.h, 8, TFT_WHITE);
  tft.setTextColor(TFT_WHITE, b.color);
  tft.setTextDatum(MC_DATUM);
  tft.drawString(labelOverride ? labelOverride : b.label, b.x + b.w / 2, b.y + b.h / 2, 2);
}

void drawHeader() {
  finger.getTemplateCount();
  tft.fillRect(0, 0, 480, 40, TFT_DARKGREY);
  tft.setTextColor(TFT_WHITE, TFT_DARKGREY);
  tft.setTextDatum(ML_DATUM);
  tft.drawString("Fingerprint System", 10, 20, 4);
  tft.setTextDatum(MR_DATUM);
  tft.drawString("Saved: " + String(finger.templateCount), 470, 20, 4);
}

// Message in the middle area; use '\n' for a second line
void showStatus(const String &msg, uint16_t color = TFT_WHITE) {
  tft.fillRect(0, 50, 480, 140, TFT_BLACK);
  tft.setTextColor(color, TFT_BLACK);
  tft.setTextDatum(MC_DATUM);
  int nl = msg.indexOf('\n');
  if (nl < 0) {
    tft.drawString(msg, 240, 110, 4);
  } else {
    tft.drawString(msg.substring(0, nl), 240, 85, 4);
    tft.drawString(msg.substring(nl + 1), 240, 135, 4);
  }
  Serial.println(msg);
}

void showHint(const String &msg) {
  tft.setTextColor(TFT_DARKGREY, TFT_BLACK);
  tft.setTextDatum(MC_DATUM);
  tft.drawString(msg, 240, 175, 2);
}

void drawMenu() {
  tft.fillScreen(TFT_BLACK);
  drawHeader();
  for (int i = 0; i < 4; i++) drawButton(i);
  showStatus("Choose an option");
}

// Returns button index or -1. Needs two consecutive hits to ignore noise.
int touchedButton() {
  uint16_t x1, y1, x2, y2;
```

```
  if (!tft.getTouch(&x1, &y1, 400)) return -1;
  delay(15);
  if (!tft.getTouch(&x2, &y2, 400)) return -1;
  if (abs((int)x1 - (int)x2) > 25 || abs((int)y1 - (int)y2) > 25) return -1;
  for (int i = 0; i < 4; i++) {
    Button &b = btns[i];
    if (x2 >= b.x && x2 <= b.x + b.w && y2 >= b.y && y2 <= b.y + b.h) return i;
  }
  return -1;
}

void waitTouchRelease() {
  uint16_t x, y;
  unsigned long t = millis();
  while (tft.getTouch(&x, &y, 200) && millis() - t < 2000) delay(20);
  delay(200);
}

bool screenTouched() {
  uint16_t x, y;
  return tft.getTouch(&x, &y, 400);
}

// ============================================================== Sensor helpers
// Result: 1 = finger captured, 0 = timeout, -1 = cancelled by touching the screen
int waitForFinger(unsigned long timeoutMs) {
  unsigned long t = millis();
  while (millis() - t < timeoutMs) {
    if (finger.getImage() == FINGERPRINT_OK) return 1;
    if (screenTouched()) { waitTouchRelease(); return -1; }
    delay(40);
  }
  return 0;
}

bool waitForRemoval(unsigned long timeoutMs) {
  unsigned long t = millis();
  while (millis() - t < timeoutMs) {
    if (finger.getImage() == FINGERPRINT_NOFINGER) return true;
    delay(40);
  }
  return false;
}

// Returns true (and prints message) if the wait did not succeed
bool waitFailed(int r) {
  if (r == 1) return false;
  showStatus(r == 0 ? "Timeout - no finger" : "Cancelled", TFT_ORANGE);
  return true;
}

// First empty slot, or 0 if full
uint16_t nextFreeId() {
  uint16_t cap = finger.capacity ? finger.capacity : 127;
  for (uint16_t id = 1; id <= cap; id++) {
    if (finger.loadModel(id) != FINGERPRINT_OK) return id;
  }
  return 0;
}

// ================================================================== Actions
void doEnroll() {
  uint16_t id = nextFreeId();
  if (id == 0) { showStatus("Sensor memory full", TFT_RED); return; }
```

```
  showStatus("Place finger\n(saving as ID " + String(id) + ")", TFT_CYAN);
  showHint("Touch the screen to cancel");
  if (waitFailed(waitForFinger(10000))) return;
  if (finger.image2Tz(1) != FINGERPRINT_OK) { showStatus("Bad image, try again", TFT_RED);
return; }

  showStatus("Remove finger", TFT_YELLOW);
  delay(800);
  if (!waitForRemoval(5000)) { showStatus("Finger not removed", TFT_ORANGE); return; }

  showStatus("Place SAME finger\nagain", TFT_CYAN);
  showHint("Touch the screen to cancel");
  if (waitFailed(waitForFinger(10000))) return;
  if (finger.image2Tz(2) != FINGERPRINT_OK) { showStatus("Bad image, try again", TFT_RED);
return; }

  if (finger.createModel() != FINGERPRINT_OK) { showStatus("Fingers did not match",
TFT_RED); return; }
  if (finger.storeModel(id) != FINGERPRINT_OK) { showStatus("Store failed", TFT_RED);
return; }

  showStatus("Enrolled!\nID = " + String(id), TFT_GREEN);
}

void doDelete() {
  showStatus("Place the finger\nyou want to delete", TFT_CYAN);
  showHint("Touch the screen to cancel");
  if (waitFailed(waitForFinger(10000))) return;
  if (finger.image2Tz() != FINGERPRINT_OK) { showStatus("Bad image, try again", TFT_ORANGE);
return; }
  if (finger.fingerFastSearch() != FINGERPRINT_OK) { showStatus("No match\nNothing deleted",
TFT_RED); return; }

  uint16_t id = finger.fingerID;
  if (finger.deleteModel(id) == FINGERPRINT_OK) showStatus("Deleted\nID " + String(id),
TFT_GREEN);
  else showStatus("Delete failed", TFT_RED);
}

void doTest() {
  showStatus("Place finger to test", TFT_CYAN);
  showHint("Touch the screen to cancel");
  if (waitFailed(waitForFinger(10000))) return;
  if (finger.image2Tz() != FINGERPRINT_OK) { showStatus("Bad image, try again", TFT_ORANGE);
return; }

  if (finger.fingerFastSearch() == FINGERPRINT_OK) {
    showStatus("MATCH  ID " + String(finger.fingerID) + "\nScore " +
String(finger.confidence), TFT_GREEN);
  } else {
    showStatus("ACCESS DENIED\nNo match", TFT_RED);
  }
}

void doEraseAll() {
  if (finger.emptyDatabase() == FINGERPRINT_OK) showStatus("All fingerprints\nerased",
TFT_GREEN);
  else showStatus("Erase failed", TFT_RED);
}

// ============================================================ Touch calibration
void runCalibration() {
  tft.fillScreen(TFT_BLACK);
```

```
  tft.setTextColor(TFT_WHITE, TFT_BLACK);
  tft.setTextDatum(MC_DATUM);
  tft.drawString("Press each corner arrow firmly", 240, 160, 2);
  delay(1500);
  tft.calibrateTouch(calData, TFT_MAGENTA, TFT_BLACK, 15);
  prefs.putBytes("cal", calData, sizeof(calData));   // save to flash
  Serial.print("calData = {");
  for (int i = 0; i < 5; i++) { Serial.print(calData[i]); if (i < 4) Serial.print(", "); }
  Serial.println("}  (saved to flash)");
}

void setupTouch() {
  prefs.begin("touchcal", false);
  bool haveCal = prefs.getBytesLength("cal") == sizeof(calData);

  if (haveCal) {
    prefs.getBytes("cal", calData, sizeof(calData));
    tft.setTouch(calData);
    // Give the user 2 seconds to ask for recalibration
    tft.fillScreen(TFT_BLACK);
    tft.setTextColor(TFT_WHITE, TFT_BLACK);
    tft.setTextDatum(MC_DATUM);
    tft.drawString("Touch the screen now", 240, 140, 4);
    tft.drawString("to recalibrate", 240, 175, 4);
    unsigned long t = millis();
    bool recal = false;
    while (millis() - t < 2000) {
      if (screenTouched()) { recal = true; break; }
      delay(30);
    }
    if (recal) {
      waitTouchRelease();
      runCalibration();
      tft.setTouch(calData);
    }
  } else {
    runCalibration();
    tft.setTouch(calData);
  }
}

// ================================================================ Arduino
void setup() {
  Serial.begin(115200);

  tft.init();
  tft.setRotation(1);               // 480 x 320 landscape
  tft.fillScreen(TFT_BLACK);

  // Fingerprint sensor
  Serial2.begin(FP_BAUD, SERIAL_8N1, FP_RX_PIN, FP_TX_PIN);
  finger.begin(FP_BAUD);
  delay(100);
  if (finger.verifyPassword()) {
    Serial.println("Sensor found");
    finger.getParameters();
  } else {
    showStatus("Sensor NOT found\nCheck TX/RX wires", TFT_RED);
    while (true) delay(1000);
  }

  setupTouch();
  drawMenu();
}
```

```
void loop() {
  // Disarm ERASE ALL after 3 seconds
  if (eraseArmed && millis() - eraseArmTime > 3000) {
    eraseArmed = false;
    drawButton(3);
    showStatus("Choose an option");
  }

  int b = touchedButton();
  if (b < 0) return;
  waitTouchRelease();

  if (b == 3) {                       // ERASE ALL needs two taps
    if (!eraseArmed) {
      eraseArmed = true;
      eraseArmTime = millis();
      drawButton(3, "TAP AGAIN");
      showStatus("Tap ERASE ALL again\nto confirm", TFT_ORANGE);
      return;
    }
    eraseArmed = false;
    doEraseAll();
  } else {
    eraseArmed = false;
    if (b == 0) doEnroll();
    if (b == 1) doDelete();
    if (b == 2) doTest();
  }

  delay(2000);                        // let the user read the result
  drawMenu();
}
```

Appendix C: Touch drawing test (used for Figures 5 to 8)

```
/*
  STEP 1: Touch-only test (no fingerprint sensor needed)
  - Calibrates touch, prints calData to Serial Monitor
  - Draw with your finger, see live X/Y, test 3 buttons
*/

#include <TFT_eSPI.h>

TFT_eSPI tft;
uint16_t calData[5];

// Button areas (landscape 480x320)
struct Btn { int x, y, w, h; const char* label; uint16_t color; };
Btn clearBtn = {  10, 270, 140, 42, "CLEAR",  TFT_BLUE };
Btn recalBtn = { 170, 270, 140, 42, "RECAL",  TFT_ORANGE };
Btn toggBtn  = { 330, 270, 140, 42, "OFF",    TFT_RED };
bool toggleState = false;

void drawBtn(Btn &b, const char* label = nullptr) {
  tft.fillRoundRect(b.x, b.y, b.w, b.h, 8, b.color);
  tft.drawRoundRect(b.x, b.y, b.w, b.h, 8, TFT_WHITE);
  tft.setTextColor(TFT_WHITE, b.color);
  tft.setTextDatum(MC_DATUM);
  tft.drawString(label ? label : b.label, b.x + b.w / 2, b.y + b.h / 2, 2);
}

bool inside(Btn &b, int x, int y) {
  return x >= b.x && x <= b.x + b.w && y >= b.y && y <= b.y + b.h;
}

void drawScreen() {
  tft.fillScreen(TFT_BLACK);
  tft.setTextDatum(TL_DATUM);
  tft.setTextColor(TFT_WHITE, TFT_BLACK);
  tft.drawString("Touch test: draw anywhere", 10, 8, 2);
  // corner markers to check accuracy
  tft.fillRect(0, 0, 12, 12, TFT_RED);
  tft.fillRect(468, 0, 12, 12, TFT_GREEN);
  tft.fillRect(0, 308, 12, 12, TFT_YELLOW);
  tft.fillRect(468, 308, 12, 12, TFT_CYAN);
  drawBtn(clearBtn);
  drawBtn(recalBtn);
  drawBtn(toggBtn, toggleState ? "ON" : "OFF");
  toggBtn.color = toggleState ? TFT_DARKGREEN : TFT_RED;
  drawBtn(toggBtn, toggleState ? "ON" : "OFF");
}

void calibrate() {
  tft.fillScreen(TFT_BLACK);
  tft.setTextColor(TFT_WHITE, TFT_BLACK);
  tft.setTextDatum(MC_DATUM);
  tft.drawString("Press each corner arrow firmly", 240, 160, 2);
  delay(1500);
  tft.calibrateTouch(calData, TFT_MAGENTA, TFT_BLACK, 15);
  tft.setTouch(calData);

  Serial.print("calData = {");
  for (int i = 0; i < 5; i++) { Serial.print(calData[i]); if (i < 4) Serial.print(", "); }
  Serial.println("}");
  Serial.println("^ copy these 5 numbers into the main sketch");
}
```

```
void setup() {
  Serial.begin(115200);
  tft.init();
  tft.setRotation(1);
  calibrate();
  drawScreen();
}

void loop() {
  uint16_t x, y;
  if (!tft.getTouch(&x, &y)) return;

  if (y > 262) {                          // button row
    if (inside(clearBtn, x, y)) { drawScreen(); }
    else if (inside(recalBtn, x, y)) { calibrate(); drawScreen(); }
    else if (inside(toggBtn, x, y)) {
      toggleState = !toggleState;
      toggBtn.color = toggleState ? TFT_DARKGREEN : TFT_RED;
      drawBtn(toggBtn, toggleState ? "ON" : "OFF");
    }
    delay(250);
    return;
  }

  tft.fillCircle(x, y, 3, TFT_YELLOW);
  tft.setTextColor(TFT_WHITE, TFT_BLACK);
  tft.setTextDatum(TR_DATUM);
  tft.drawString("X:" + String(x) + " Y:" + String(y) + "   ", 470, 24, 2);
  Serial.printf("x=%d y=%d\n", x, y);
}
```

Appendix D: Raw touch test (used to find Error 3)

Open the Serial Monitor at 115200. Press and release the screen and watch whether x, y and pressure change.

```
#include <TFT_eSPI.h>
TFT_eSPI tft;

void setup() {
  Serial.begin(115200);
  tft.init();
  tft.setRotation(1);
  tft.fillScreen(TFT_BLACK);
  tft.setTextColor(TFT_WHITE, TFT_BLACK);
  tft.drawString("Press the screen firmly", 20, 20, 4);
}

void loop() {
  uint16_t x, y;
  uint16_t z = tft.getTouchRawZ();
  tft.getTouchRaw(&x, &y);
  Serial.printf("raw x=%d  y=%d  pressure=%d\n", x, y, z);
  delay(200);
}
```